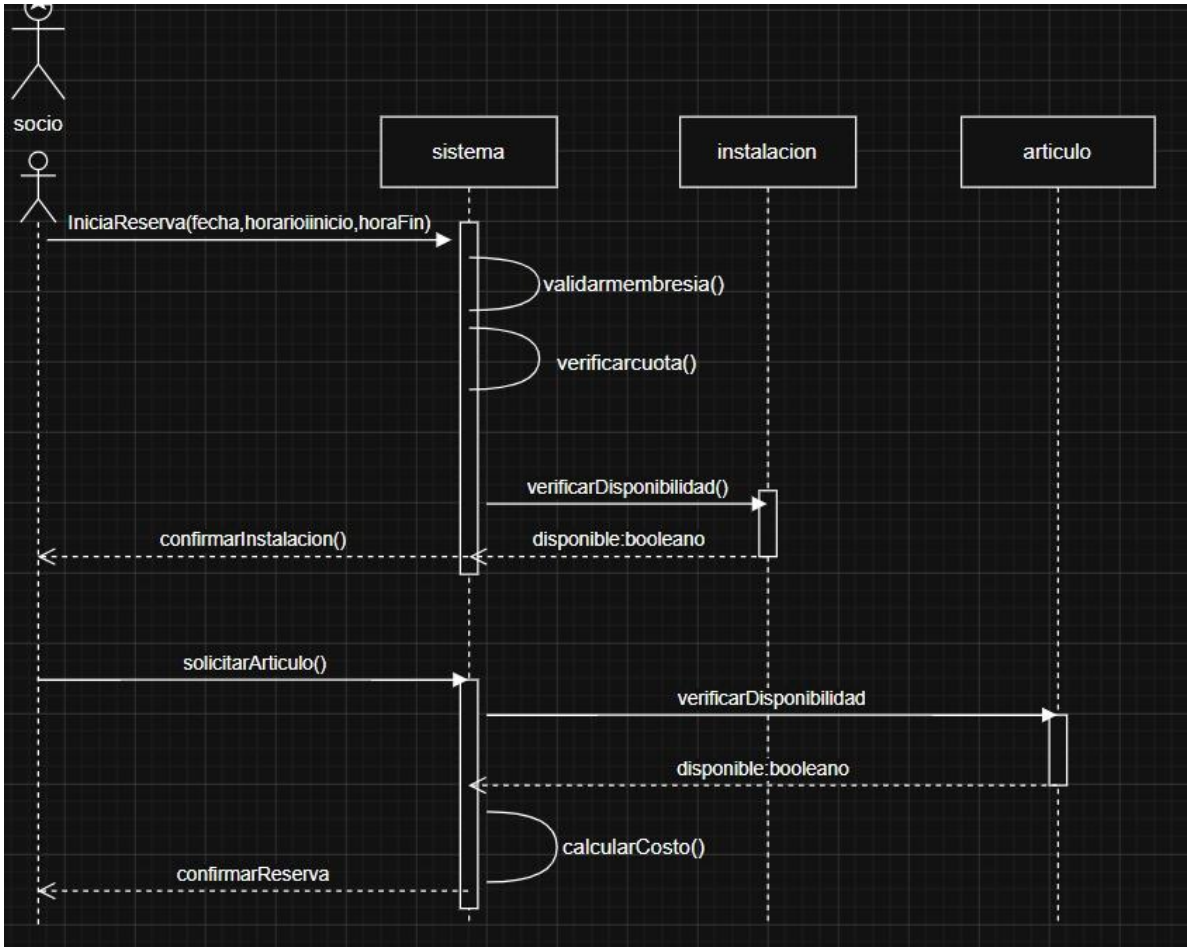
Justification for the sequence diagram

Sports center booking system



1. The problem being modeled

The sports center offers various facilities (swimming pools, fronton courts, gyms, and tennis courts). Information regarding the member—such as name, address, city, state, phone number, and membership fee—is stored. Additionally, items such as balls, nets, and rackets are available for reservation if needed.

Each facility is reserved by a member for a specific date, with a start time and an end time. A single reservation can include one or more pieces of sports equipment, which are rented separately. For example, playing tennis requires reserving the court and, if necessary, the rackets as well.

This sequence diagram illustrates that exact process: the complete, step-by-step workflow of how a member makes a reservation.

2. Why a sequence diagram was chosen

A sequence diagram is used to show the chronological order in which different system elements communicate with one another to complete a process. It is the right tool here because the statement describes an action (making a reservation), not just data.

• A class diagram shows data structure (the attributes of each entity) but does not indicate the order in which events occur.

• An entity-relationship diagram shows how database tables are related but does not represent a process flow.

• A sequence diagram does answer the question actually posed by the statement: “What steps does the system follow when someone makes a reservation?”

3. The participants: why these and not others

Partner (actor): Represented by a stick figure because it is an actor—that is, an entity external to the system using it. It is the party that initiates the entire process.

System: Represents the software that coordinates everything. Most of the decision-making logic (validation, calculation, confirmation) resides here, as it is the central component connecting the partner to the center's resources.

Facility: Modeled as a separate object because it has its own independent state: its availability schedule. The System needs to “ask” the facility if it is free, so it must be a distinct participant.

Sports equipment: For the same reason as the facility: it has its own inventory and availability (e.g., how many rackets are available at a given time), and the problem statement indicates that it is rented “separately”—meaning it involves its own verification process.

4. Overview and rationale for each message

The following table explains the origin of each step in the diagram and the reason for its specific form (solid arrow, dashed arrow, or self-message).

Mensaje	**Justificación (por qué existe)**	**Notación usada**
StartReservation(date, startTime, endTime)	The statement reads: “each facility is reserved for a specific date, from a start time to an end time.” These three pieces of information must be captured in the initial message because they form the basis for everything that follows.	Synchronous message (solid arrow) from the Partner actor to the System object.
validateMembership()	The statement indicates that the system stores member data, including their membership fee. Before allowing a reservation, the system must confirm that the person requesting it is indeed a registered member.	Self-message (the arrow exits and returns to the same activation bar) because it is an internal system task; it

		does not involve another object.
verifyQuota()	It is separate from membership validation because they are two distinct checks: one confirms that the member exists, and the other confirms that they are up to date with their payments.	Self-message, same as the previous one.
selectInstallation()	The center offers various facilities (swimming pools, fronton courts, gyms, tennis courts). Members must indicate which facility they wish to use before availability can be checked.	Self-message or internal System step that processes the selection already submitted in the initial message.
checkAvailability() (to Installation)	Although the statement does not say so explicitly, it is a necessary logical consequence: if every reservation has a date and time, the system must necessarily verify that no one else has that facility occupied at that same time.	Synchronous message from the System to the Installation object; activates its activation bar.
available : boolean	It is the mandatory response to the previous question; without it, the System could not decide whether to proceed with or reject the reservation.	Return message (dotted arrow) from Installation to System.
confirmInstallation()	This concludes the first stage of the process: it informs the member that that part of their reservation (the facility) has been secured.	Return message from the system to the Partner actor.
requestArticle()	The statement explicitly says: “a reservation may include one or more pieces of sports equipment,” and gives the example of tennis, where, in addition to the court, one can request a racket.	Synchronous message from the Partner to the System; it initiates a new activation because it is a second, independent request.
checkAvailability() (on Item)	The same reasoning applies as with the facility: the items are physical, finite goods (there is only a certain number of rackets), so one must also verify their availability.	Synchronous message from the System to the Sports Equipment object.
available : boolean	Response equivalent to that for the installation, but for the requested item.	Return message from Sports Item to System.
calculateCost()	The statement mentions a membership fee and items that are rented “separately,” implying that each element has an associated cost that must be added up before confirming.	System auto-message.
registerReservation()	A confirmed reservation must be stored so that it can be consulted later; this is a necessary step for the system to function, even if the statement does not spell it out word for word.	System auto-message.
confirmReservation()	It is the natural conclusion of the process: the member needs final confirmation that their complete reservation (facility + items) has been made.	Return message from the system to the Partner actor.

5. Conclusion:

In conclusion, the diagram effectively illustrates the step-by-step reservation process, though some details could be refined. For instance, I need to adjust the elongated rectangles along the dotted lines so they precisely indicate when the system or objects are active. Additionally, I would like to include options for error scenarios—such as an expired installment—and correct the spelling of certain names to ensure the diagram is neater and more accurate.